Fichas bibliográficas: fundamentos de sistemas distribuidos

Bryan Humberto Lombeida Escaleras

6 de noviembre de 2026

1. Fichas bibliográficas

1.1. Rendimiento de microservicios

DOI verificado: `10.1007/s10664-021-10088-0`
Resumen (incluye dato concreto): El artículo analiza por qué microservicios independientes pierden rendimiento ante cambios de carga. Camilli y Russo modelan temporalmente las violaciones de requisitos con funciones de teoría del crecimiento. Validan el método en Sock Shop, comercio experimental, y en un sistema de telecomunicaciones de Ericsson en producción. Los modelos no lineales acotados, sobre todo curvas S, describen servicios que se recuperan; las tendencias lineales indican degradaciones persistentes. Diez de doce microservicios del sistema de Ericsson presentaron violaciones de rendimiento (sección 5.4, tabla 7). Permite distinguir servicios de aquellos que necesitan atención para mantener rendimiento [1].
Nota personal: Ahora entiendo que un servicio puede responder lentamente y aun así recuperarse. Me parece útil observar la evolución del problema antes de decidir si necesita más recursos o una corrección.

1.2. Ecosistema de reutilización

DOI verificado: `10.1109/MS.2022.3167447`
Resumen (incluye dato concreto): El artículo describe cómo Ericsson reutiliza microservicios entre aplicaciones manteniendo coherencia en configuración y evolución. Los autores presentan Application Development Platform, sustentada en un catálogo interno, reglas de diseño y comprobaciones automáticas. Sus canalizaciones de integración y entrega continua conectan servicios y aplicaciones, usando Spinnaker para coordinar versiones y Helm

para desplegar. La experiencia indica que los equipos deben descubrir componentes, conocer su madurez y colaborar con sus mantenedores. El ecosistema ADP comenzó a finales de 2017 (página 69). El caso demuestra que compartir servicios requiere coordinación, reglas claras y herramientas comunes entre equipos [2].
Nota personal: Me llamó la atención que reutilizar un microservicio implica mucho más que compartir código. También hay que facilitar que otros equipos lo encuentren, comprendan sus reglas y puedan mejorarlo.

1.3. Observabilidad de aplicaciones web

DOI verificado: `10.1016/j.jss.2026.112886`
Resumen (incluye dato concreto): El estudio aborda la dificultad de observar aplicaciones web cuando los registros se añaden manualmente. En Berger-Levrault, esa práctica generaba datos inconsistentes y exigía mantenimiento. Los investigadores diseñan una arquitectura de observabilidad y amplían agentes de OpenTelemetry para recoger eventos y trazas sin modificar el código. Evalúan utilidad y coste de instrumentar aplicaciones industriales. Resultados indican que aporta información para analizar la experiencia de usuario y apoyar pruebas automáticas con baja sobrecarga. Se evaluaron cuatro aplicaciones web industriales de diferentes tecnologías (sección 3). Facilita decisiones [3].
Nota personal: Esta lectura me hizo ver que los registros por sí solos pueden dejar preguntas sin respuesta. Entiendo mejor por qué reunir trazas y eventos ayuda a conocer lo que realmente ocurre durante el uso.

1.4. Diagnóstico de fallos

DOI verificado: `10.1109/TSC.2023.3290018`
Resumen (incluye dato concreto): El artículo estudia cómo localizar el origen de un fallo cuando microservicios conectados presentan síntomas relacionados. Los autores afirman que examinar solo métricas, registros o trazas limita el diagnóstico. Proponen DiagFusion, que representa conjuntamente esas fuentes, amplía muestras para compensar fallos poco frecuentes y construye un grafo de dependencias con datos de despliegue y trazas. Una red neuronal de grafos identifica la instancia causante y clasifica el tipo de fallo. En conjuntos de datos reales y variados, DiagFusion mejoró la localización de causas entre 20,9 % y 368 %, y la clasificación del fallo entre 11 % y 169 % (resumen, página 3851) [4].
Nota personal: Aprendí que el servicio donde aparece un error no siempre es el que lo causó. Combinar distintas señales me parece una forma más fiable

de seguir la cadena del fallo.

1.5. Reutilización con CI/CD e InnerSource

DOI verificado: 10.1007/s10664-024-10595-w
Resumen (incluye dato concreto): La investigación analiza beneficios y dificultades de compartir microservicios entre productos y equipos de Ericsson. Mediante un estudio de caso en dos fases —entrevistas, discusiones y portales internos, y consulta posterior sobre mejoras aplicadas— identifica costes de aprendizaje, integración y cumplimiento, y presión sobre los equipos responsables. Ericsson responde fomentando contribuciones internas con InnerSource y automatizando comprobaciones de reglas de diseño. El catálogo ADP Marketplace alojaba más de 280 microservicios usados por más de 70 equipos (sección 4.2). Compartir componentes exige apoyo constante, automatización y colaboración entre equipos [5].
Nota personal: Me parece importante considerar el esfuerzo que exige mantener componentes compartidos. La reutilización aporta valor cuando los equipos también pueden contribuir y comprobar los cambios con confianza.

Referencias

[1] M. Camilli y B. Russo, «Modeling Performance of Microservices Systems with Growth Theory,» *Empirical Software Engineering*, vol. 27, 2022, art. 39. DOI: 10.1007/s10664-021-10088-0

[2] M. Usman, D. Badampudi, C. Smith y H. Nayak, «An Ecosystem for the Large-Scale Reuse of Microservices in a Cloud-Native Context,» *IEEE Software*, vol. 39, n.º 5, págs. 68-75, 2022. DOI: 10.1109/MS.2022.3167447

[3] A. Shatnawi, B. Rima, Z. Alshara, G. Darbord, A.-D. Seriai y C. Bortolaso, «Observability of web applications: An industrial case study,» *Journal of Systems and Software*, vol. 239, 2026, art. 112886. DOI: 10.1016/j.jss.2026.112886

[4] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang y S. Xia, «Robust Failure Diagnosis of Microservice System Through Multimodal Data,» *IEEE Transactions on Services Computing*, vol. 16, n.º 6, págs. 3851-3864, 2023. DOI: 10.1109/TSC.2023.3290018

[5] D. Badampudi, M. Usman y X. Chen, «Large scale reuse of microservices using CI/CD and InnerSource practices – a case study,» *Empirical Software Engineering*, vol. 30, n.º 2, 2025, art. 41. DOI: 10.1007/s10664-024-10595-w

Declaración de uso de inteligencia artificial

Utilicé Codex como apoyo para resolver problemas técnicos de LaTeX, organizar las citas y ajustar la redacción de mis borradores al límite de 100 palabras por resumen. Codex también propuso formulaciones para las notas personales. Los DOI, los datos bibliográficos y las cifras se contrastaron con las fuentes originales. La selección de las ideas y la revisión final de los resúmenes y las notas personales corresponden al autor.